

SUKKUR IBA UNIVERSITY

Department of Computer Science

Web Engineering (CSE-311)

Lab Report 05: JavaScript Fundamentals

Pure Node.js, Variables, Scoping, Array Pipelines, Objects & Validated Bill Calculator

Student Name:	Muhammad Tayyab
CMS ID:	023-24-0118
Course:	Web Engineering (CSE-311)
Semester:	5th Semester (Fall 2026)
Submitted To:	Lab Instructor
Repository:	github.com/taybro-o/web-engineering-lab

1. Warm-up: Quick Recall (warmup.js)

In warmup.js, I created a starting JavaScript file executed directly in Node.js without browser overhead:

- **Variables:** Declared const name = "Muhammad Tayyab" and let score = 0.
- **Function:** Defined double(n) returning n * 2.
- **Array:** Created an array of skills and logged the second element at index 1.

Rule: JavaScript execution in Node.js allows running pure command-line scripts without DOM or window dependencies.

File: Week 5/warmup.js

```
// warmup.js
const name = "Muhammad Tayyab";
let score = 0;

function double(n) {
  return n * 2;
}

const skills = ["JavaScript", "CSS Layout", "HTML5 Semantics"];

console.log(`Student Name: ${name}`);
console.log(`Initial Score: ${score}`);
console.log(`Double of 21: ${double(21)}`);
console.log(`Second Skill (index 1): ${skills[1]}`);
```

2. Class Activity A: const / let / === Drill (basics.js)

I investigated scoping, variable mutability, and JavaScript type coercion rules:

- **const Reassignment:** Attempting to reassign course = "Other" threw a TypeError: Assignment to constant variable.
- **let Increment:** let count = 0 was successfully incremented twice to 2.
- **Equality Drill:**
 - 0 == false is **true**: Loose equality (==) coerces boolean false to number 0.
 - 0 === false is **false**: Strict equality (===) compares without type coercion (number vs boolean).
 - [] == false is **true**: Empty array [] coerces to primitive string "", which coerces to number 0.

Rule: Always use const by default, let when reassigning, and never var. Always use ===.

File: Week 5/basics.js (Activity A)

```
// basics.js - Activity A
const course = "Web Eng";
try {
  course = "Other"; // Throws TypeError
} catch (err) {
  console.log(`Attempted reassignment error: ${err.name} - ${err.message}`);
}

let count = 0;
count++;
count++; // count is now 2

console.log(0 == false); // true: false coerces to 0
console.log(0 === false); // false: number !== boolean
console.log([] == false); // true: [] -> "" -> 0
```

3. Class Activity B: String Utilities & Arrow Functions (basics.js)

I implemented three utility functions utilizing arrow syntax and native string methods:

- **getInitials("Ayesha Khan")**: Splits string by whitespace, maps each word to its uppercase first char, and joins them to return "AK".
- **isPalindrome("Race car")**: Sanitizes non-alphanumeric characters, converts to lowercase, reverses, and verifies equality (returns true).
- **countWords("the quick brown fox")**: Trims whitespace and splits on regular expression `/s+/` to accurately count words (returns 4).

File: Week 5/basics.js (Activity B)

```
// basics.js - Activity B
const getInitials = (fullName) => {
  return fullName.trim().split(/\s+/).map(p => p[0].toUpperCase()).join("");
};

const isPalindrome = (str) => {
  const clean = str.toLowerCase().replace(/^[a-z0-9]/g, "");
  return clean.length > 0 && clean === clean.split("").reverse().join("");
};

const countWords = (sentence) => {
  const trimmed = sentence.trim();
  return trimmed.length === 0 ? 0 : trimmed.split(/\s+/).length;
};
```

4. Class Activity C: Conditionals & Nullish Coalescing (basics.js)

I compared logical branching and safe property fallback mechanisms:

- **letterGrade(score)**: Implemented standard grade classification (A: ≥ 90 , B: ≥ 80 , C: ≥ 70 , D: ≥ 60 , F: < 60) using if / else if.
- **Nullish Coalescing (??)**: `displayCount(n)` returns `n ?? "No count provided"`. When tested with 0, it correctly returns 0 because 0 is a valid number and neither null nor undefined. In contrast, `||` would have incorrectly treated 0 as falsy.

File: Week 5/basics.js (Activity C)

```
// basics.js - Activity C
const letterGrade = (score) => {
  if (score >= 90) return "A";
  else if (score >= 80) return "B";
  else if (score >= 70) return "C";
  else if (score >= 60) return "D";
  else return "F";
};

const displayCount = (n) => n ?? "No count provided";

console.log(displayCount(0)); // 0 (preserved!)
console.log(displayCount(null)); // "No count provided"
console.log(displayCount(42)); // 42
```

5. Class Activity D: Students Pipeline with Array Methods (arrays.js)

I processed a dataset of student records using pure, non-mutating functional array pipelines:

- **Class Average**: Calculated using `students.reduce((sum, s) => sum + s.score, 0) / students.length` (76.33).
- **Above-Average Names**: Chained `.filter(s => s.score > avg).map(s => s.name)` yielding ['Ali', 'Sara', 'Zainab', 'Bilal'].
- **Predicates**: Evaluated `students.some(s => s.score > 90)` (true) and `students.every(s => s.score > 40)` (true).
- **Immutable Sorting**: Sorted a cloned array using `[...students].sort((a, b) => b.score - a.score)` without mutating the original list.

File: Week 5/arrays.js

```
// arrays.js - Activity D
const students = [
  { name: "Ali", score: 82 }, { name: "Sara", score: 95 },
  { name: "Ahmed", score: 68 }, { name: "Zainab", score: 77 },
  { name: "Bilal", score: 91 }, { name: "Hamza", score: 45 }
];

const total = students.reduce((acc, s) => acc + s.score, 0);
const avg = total / students.length;

const aboveAvg = students.filter(s => s.score > avg).map(s => s.name);
const hasTopScorer = students.some(s => s.score > 90);
const allPassThreshold = students.every(s => s.score > 40);
const sorted = [...students].sort((a, b) => b.score - a.score);
```

6. Class Activity E: Objects & groupBy via reduce (objects.js)

I manipulated product records using destructuring, spread operations, and category bucketing:

- **In-Stock Filter:** Filtered available products and mapped titles: `products.filter(p => p.inStock).map(p => p.title)`.
- **Inventory Valuation:** Computed aggregate catalog cost via `reduce` (PKR 300).
- **groupBy:** Implemented a generic categorization reducer folding the flat product array into grouped dictionary lists keyed by category.
- **Immutability via Spread:** Demonstrated updating product price via `{ ...firstProduct, price: 70 }` without mutating original state.

File: Week 5/objects.js

```
// objects.js - Activity E
const products = [
  { id: 1, title: "Mechanical Keyboard", price: 85, category: "peripherals", inStock: true },
  { id: 2, title: "USB-C Fast Cable", price: 15, category: "cables", inStock: true },
  { id: 3, title: "Wireless Mouse", price: 55, category: "peripherals", inStock: false },
  { id: 4, title: "HDMI 2.1 Cable", price: 25, category: "cables", inStock: true },
  { id: 5, title: "ANC Headphones", price: 120, category: "audio", inStock: true }
];

const groupBy = (items, key) => items.reduce((acc, item) => {
  const k = item[key];
  if (!acc[k]) acc[k] = [];
  acc[k].push(item);
  return acc;
}, {});
```

7. Class Activity F & Homework Task 4: Tip Calculator (calculator.js)

I authored a production-ready restaurant bill calculator function with defensive parameter validation:

- **Validation:** Checked that amount is a positive number, tipPercent is non-negative, and people is an integer ≥ 1 . Throws descriptive errors if invalid.
- **Safe try/catch:** Wrapped failing invocations (negative bill, people: 0, non-number tip) inside try/catch blocks, allowing tests to run and log error messages without crashing.
- **formatBill Helper:** Created a multi-line formatted receipt printing clear padded financial breakdowns.

File: Week 5/calculator.js

```
// calculator.js - Activity F & Homework Task 4
const calculateBill = ({ amount, tipPercent = 15, people = 1 } = {}) => {
  if (typeof amount !== "number" || amount <= 0) throw new Error("Invalid amount");
  if (typeof tipPercent !== "number" || tipPercent < 0) throw new Error("Invalid tip");
  if (!Number.isInteger(people) || people < 1) throw new Error("Invalid people");

  const tipAmount = amount * (tipPercent / 100);
  const totalAmount = amount + tipAmount;
  return {
    amount: Number(amount.toFixed(2)),
    tipPercent: Number(tipPercent.toFixed(2)),
    tipAmount: Number(tipAmount.toFixed(2)),
    totalAmount: Number(totalAmount.toFixed(2)),
    people,
    perPerson: Number((totalAmount / people).toFixed(2))
  };
};
```

8. In-Class Lab Consolidated Script (lab.js)

As specified in the lab completion checklist, I unified all components into a polished lab.js script. It satisfies all five checklist requirements: string utilities, array pipeline without for-loops, object grouping via reduce, defensive tip calculation with tested error handling, and strict equality / nullish operators. Running node lab.js completes with zero uncaught exceptions and pristine output.

File: Terminal Output: node lab.js

```
/* lab.js verification output */
> node lab.js
--- 1. String Utilities ---
getInitials("Muhammad Tayyab"): MT
isPalindrome("Race car"): true
countWords("the quick brown fox"): 4

--- 2. Students Pipeline ---
Class Average: 76.33
Students scoring above class average: [ 'Ali', 'Sara', 'Zainab', 'Bilal' ]

--- 3. Inventory Categorization ---
Total inventory price: PKR 300
Categories: peripherals (2 items), cables (2 items), audio (1 items)

--- 4. Bill Calculator & Error Handling ---
Happy Path Bill: PKR 2640.00 split 3 ways = PKR 880.00 each
Caught Error 1: Amount must be a positive number.
Caught Error 2: People count must be an integer of at least 1.
Caught Error 3: Tip percentage must be a non-negative number.
```

9. Homework Task 1: Gradebook Script (gradebook.js)

I wrote gradebook.js to analyze an array of student objects using strictly functional array methods without classic loops:

- **Class Average:** 73.00 (via reduce).
- **Highest Scorer:** Muhammad Tayyab (94 marks) (via reduce).
- **Lowest Scorer:** Dua Malik (42 marks) (via reduce).
- **Pass Rate (%):** 75.00% (6 out of 8 students passing threshold ≥ 50 via filter).
- **Sorted Ranking:** Cloned and sorted high to low via [...gradebook].sort().

File: Week 5/gradebook.js

```
// gradebook.js - Homework Task 1
const gradebook = [
  { name: "Muhammad Tayyab", score: 94 },
  { name: "Ayesha Khan", score: 88 },
  { name: "Bilal Ahmed", score: 76 },
  { name: "Zainab Fatima", score: 91 },
  { name: "Hamza Tariq", score: 48 },
  { name: "Sara Noor", score: 83 },
  { name: "Usman Ali", score: 62 },
  { name: "Dua Malik", score: 42 }
];

const avg = gradebook.reduce((sum, s) => sum + s.score, 0) / gradebook.length;
const highest = gradebook.reduce((max, s) => s.score > max.score ? s : max);
const lowest = gradebook.reduce((min, s) => s.score < min.score ? s : min);
const passRate = (gradebook.filter(s => s.score >= 50).length / gradebook.length) * 100;
const ranked = [...gradebook].sort((a, b) => b.score - a.score);
```

10. Homework Task 2: Predict Before You Run (predict.txt / predict.js)

I documented predictions, executed JavaScript code tests, and analyzed coercion quirks:

- `0 == false`: Evaluates to **true** because loose equality coerces false to number 0.
- `[] == false`: Evaluates to **true** because `[]` is converted to primitive string `""`, which coerces to 0.
- `[] == ![]`: Evaluates to **true** because `![]` evaluates to false first (objects are truthy), then `[] == false` evaluates to true.
- `0 || 42 vs 0 ?? 42`: Evaluates to **42 vs 0**. `||` considers 0 falsy, whereas `??` preserves 0 because it is not null/undefined.
- `[10, 2, 5, 1].sort()`: Results in **[1, 10, 2, 5]** because default sorting converts to strings and compares UTF-16 code units lexicographically.
- **Shallow Spread**: Mutating nested properties in an object cloned via spread `{ ...obj }` also mutates the original object because nested references are shared.

File: Week 5/predict.txt

```
/* predict.js sample output */
Test 1: 0 == false      | Predicted: true      | Actual: true
Test 2: 0 === false    | Predicted: false    | Actual: false
Test 3: [] == false    | Predicted: true     | Actual: true
Test 4: [] == ![]     | Predicted: true     | Actual: true
Test 5: null == undefined | Predicted: true     | Actual: true
Test 6: typeof NaN     | Predicted: 'number' | Actual: 'number'
Test 7: 0 || 42 vs 0 ?? 42 | Predicted: 42 vs 0 | Actual: 42 vs 0
Test 8: [10, 2, 5].sort() | Predicted: [1,10,2,5] | Actual: [1,10,2,5]
```

11. Homework Task 3: freeCodeCamp Practice Verification (freecodecamp.md)

Completed the **Basic JavaScript** section (113 interactive coding challenges) within the freeCodeCamp JavaScript Algorithms and Data Structures curriculum. The exercises provided rigorous practice in variables, scopes, data structures, conditional chains, nested objects/arrays, loops, recursion, and ternary operators, reinforcing the fundamentals taught in class.